

8-step Python Decorators Exercises

What you will explore

Complete these exercises sequentially to review syntax wrappers, parameter forwarding, metadata preservation, decorator nesting, and stateful decorator factories.

Exercise 1: String Uppercase Decorator (Syntax Warm-up)

Objective: Understand the basic syntax and structure of a null-decorator wrapper.

Instructions: Write a basic decorator named `uppercase_decorator` that wraps any function returning a string, converting its output to uppercase. Verify that the decorator doesn't crash when acting on functions returning non-string datatypes.

```
# Exercise 1: Basic Uppercase Decorator Template
# Task: Write a basic decorator that converts the string output of a function to uppercase.

from functools import wraps

def uppercase_decorator(func):
    """Converts the returned string value of the target function to uppercase."""
    # TODO: Implement the decorator wrapper closure
    pass

# Sample usage:
@uppercase_decorator
def get_welcome_message(name):
    """Returns a personalized welcome message string."""
    return f"Welcome, {name}! Let's learn decorators."

if __name__ == "__main__":
    result = get_welcome_message("delegate")
    print(f"Result: {result}")
    print(f"Preserved Docstring: {get_welcome_message.__doc__}")
```

Exercise 2: Double Invocation Decorator (Execution Control)

Objective: Learn how to manipulate the target function's execution boundaries.

Instructions: Write a decorator named `double_invocation` that executes the decorated target function twice on every invocation. The decorator should return the return value of the SECOND execution.

```
# Exercise 2: Double Invocation Decorator Template
# Task: Create a decorator that executes the decorated target function twice on every call.

from functools import wraps

def double_invocation(func):
    """Executes the target function twice, returning the result of the second execution."""
    # TODO: Execute func twice and return second output
    pass

# Sample usage:
@double_invocation
def send_alert(message):
    """Simulates sending an alert message."""
    print(f"ALERT: {message}")
    return "Alert Sent"

if __name__ == "__main__":
    status = send_alert("Database connection warning!")
    print(f"Final Status: {status}")
```

Exercise 3: Positional Argument Type Validator (Input Guard)

Objective: Enforce typing rules dynamically before executing inner business logic.

Instructions: Write a decorator named `type_check_string` that validates that the first positional argument passed to the decorated function is of type `str`. If not, raise a `TypeError` with an informative message.

```
# Exercise 3: Type Validation Decorator Template
# Task: Write a decorator that validates the datatype of a function's arguments.

from functools import wraps

def type_check_string(func):
    """Validates that the first positional argument passed to the function is a string."""
    # TODO: Verify first positional argument args[0] is of type str
    pass

# Sample usage:
@type_check_string
def process_username(username):
    """Processes and registers a unique username."""
    print(f"Username '{username}' has been successfully processed.")
    return True

if __name__ == "__main__":
    try:
        process_username("alex_python") # Success
        process_username(12345)         # Raises TypeError
    except TypeError as e:
        print(f"Caught expected error: {e}")
```

Exercise 4: Execution Logger (Preserving Metadata)

Objective: Prevent metadata loss during decoration using `functools.wraps`.

Instructions: Create an `execution_logger` decorator that prints logging headers upon entering and exiting the function. Use `functools.wraps` to ensure that function docstrings and names are preserved after decoration.

```
# Exercise 4: Execution Logger with Metadata Preservation Template
# Task: Create a decorator that logs function metadata and parameters cleanly.

from functools import wraps

def execution_logger(func):
    """Logs the entry, exit, and parameters of the decorated function while preserving attributes."""
    # TODO: Use @wraps(func) to protect original docstring attributes
    pass

# Sample usage:
@execution_logger
def compute_area(length, width):
    """Calculates the area of a rectangle."""
    return length * width

if __name__ == "__main__":
    print(f"Docstring: {compute_area.__doc__}")
    compute_area(5, 10)
```

Exercise 5: Performance Monitor Stacking (Nesting Order)

Objective: Observe and trace the nested execution flow of multiple stacked decorators.

Instructions: Stack a stateful `@count_calls` decorator and a `@timer` decorator together. Trace the execution flow and explain why the stacking order affects what values are timed and counted.

```
# Exercise 5: Performance Monitor Stacking Template
# Task: Stack multiple decorators to measure call count and execution durations simultaneously.

import time
from functools import wraps

def count_calls(func):
    """Tracks the total number of times a function has been executed."""
    # TODO: Implement stateful call counter attribute
    pass

def timer(func):
    """Measures and prints the execution duration of a function call."""
    # TODO: Measure and log time.perf_counter() elapsed delta
    pass

# Stack decorators: timer on top, count_calls on bottom
@timer
@count_calls
def heavy_computation(n):
    """Simulates a heavy mathematical calculation loop."""
    total = 0
    for i in range(n):
        total += i * i
    return total

if __name__ == "__main__":
    heavy_computation(100000)
    heavy_computation(500000)
```

Exercise 6: Stateful Retry Decorator (Configurable Factories)

Objective: Build a configurable three-tier decorator factory that accepts runtime parameters.

Instructions: Create a decorator factory named `@retry(max_attempts, delay_seconds)`. If the target function raises any Exception, sleep for the specified duration and retry execution up to `max_attempts`.

```
# Exercise 6: Stateful Retry Decorator Factory Template
# Task: Write a configurable decorator factory that retries failing operations.

import time
from functools import wraps

def retry(max_attempts=3, delay_seconds=1):
    """A decorator factory that retries execution on exception failures up to max_attempts."""
    # TODO: Implement three-tier factory structure
    pass

# Sample usage:
@retry(max_attempts=3, delay_seconds=0.5)
def unstable_network_call(url):
    """Simulates an unstable HTTP endpoint call."""
    import random
    if random.random() > 0.3:
        raise ConnectionError("Timeout attempting to resolve endpoint.")
    return f"Response received from {url}!"

if __name__ == "__main__":
    try:
        msg = unstable_network_call("https://api.python-decorators.academy")
        print(f"Success Message: {msg}")
    except ConnectionError as e:
        print(f"Operation aborted after all retries: {e}")
```

Exercise 7: Sliding Window Rate Limiter (Information Hiding)

Objective: Incorporate stateful closure boundaries into decorators for secure rate limiting.

Instructions: Write a decorator factory named `@limit_rate(max_calls, period_seconds)` that tracks invocation timestamps inside its enclosing scope, raising a `RuntimeError` if thresholds are exceeded.

```
# Exercise 7: Token Bucket Rate Limiter Template
# Task: Build a stateful, rate-limiting decorator factory using closure states.

import time
from functools import wraps

def limit_rate(max_calls=5, period_seconds=2):
    """Decorator factory that limits calls to a function within a specific time window."""
    # TODO: Manage enclosing timestamps sliding window array
    pass

# Sample usage:
@limit_rate(max_calls=3, period_seconds=2)
def read_secure_vault():
    """Simulates retrieving highly sensitive credentials."""
    return "Vault Data"

if __name__ == "__main__":
    # Test rate limits
    for i in range(5):
        try:
            print(f"Attempt {i+1}: {read_secure_vault()}")
            time.sleep(0.2)
        except RuntimeError as e:
            print(f"Attempt {i+1} Blocked: {e}")
```

Exercise 8: Advanced Memoizer with Shared Cache Store

Objective: Construct a high-performance optimization decorator with modular cache state sharing.

Instructions: Write a `@memoize(custom_cache_store)` decorator factory. It should map function argument tuples to computed results. If a custom cache dictionary is passed, use it, enabling multiple decorated targets to share state.

```
# Exercise 8: Configurable Memoization Cache Template
# Task: Build an advanced memoization decorator that supports a shared, custom caching dictionary.

from functools import wraps

def memoize(custom_cache_store=None):
    """Advanced memoization decorator factory that supports an optional custom storage container."""
    # TODO: Construct a cache dictionary and support shared injection
    pass

shared_database = {}

@memoize(custom_cache_store=shared_database)
def fibonacci(n):
    """Calculates the nth Fibonacci number recursively with memoization."""
    if n < 2:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

if __name__ == "__main__":
    # Warm up cache
    print(f"Fibonacci(30): {fibonacci(30)}")
    print(f"Shared Database Cache Size: {len(shared_database)}")
```